SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACTIVE LEARNING METHODOLOGY REPORT

SQL INJECTION AND SECURE DATABASE INTERACTION

AJAY.I – 71812401007

THIRD YEAR B.E. CSE – V SEM

Academic Year 2026-2027

20CS255 & SECURITY IN ENGINEERING

COURSE COORDINATOR
Ms.A.Ishwarya,AP/CSE

SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

20CS212 & WEB TECHNOLOGIES

Assessment

INDEX

S. No.	Name & Roll No.	Scenario / problem statement (2 Marks)	Identification and explanation of security vulnerability (3 Marks)	Selection of suitable security mechanism (3 Marks)	Justification / application to the scenario (2 Marks)	Total (10)
1.	AJAY.I - 71812401007					

SIGNATURE OF COURSE COORDINATOR
Ms.A.Ishwarya,AP/CSE

CONTENTS

S. No.	TITLE	PAGE NUMBER
1.	Introduction	1
2.	Scenario / problem statement	3
3.	Identification and explanation of security vulnerability	5
4.	Selection of suitable security mechanism	8
5.	Justification / application to the scenario	10
6.	Expected Output	13
7.	Conclusion	15

1. INTRODUCTION

Almost every banking website is a thin layer of web pages placed in front of a relational database. Customer profiles, account balances, beneficiaries, card details and transaction history are all stored in tables, and the web application reaches them by sending Structured Query Language (SQL) statements to the database server. Even the simplest action, logging in, is a database query: the application asks the database whether a customer with the given username and password exists.

The security problem begins when the application builds that SQL statement by joining fixed program text with text typed by the customer. SQL is a language, and the database cannot tell which part of a statement was written by the developer and which part came from the login form. If the input contains characters that have meaning in SQL, such as a single quote, a comment marker or a keyword like OR, the input stops being a value and becomes part of the command. This class of flaw is called SQL injection (SQLi).

SQL injection has been known for more than twenty-five years, yet it remains one of the most damaging web vulnerabilities. Injection is listed as A03 in the OWASP Top 10 (2021), and CWE-89, “Improper Neutralization of Special Elements used in an SQL Command”, consistently appears in the MITRE CWE Top 25 Most Dangerous Software Weaknesses. For a bank the consequences are severe: an attacker may log in as another customer without a password, read the entire customer table, alter balances or delete transaction records.

SQL injection is also an access-control failure. The web application connects to the database with an account that holds real privileges. When an attacker's text is executed as SQL, it runs with those privileges, so a stranger on the internet effectively borrows the identity of the application. The principles of least privilege, complete mediation and a clear separation between code and data, which govern privileged programs in an operating system, apply equally here.

This report analyses the SQL injection weakness in the login function of a banking website, explains step by step how crafted input changes the query, selects a layered set of defences centred on parameterized queries, justifies each defence against the scenario, and describes the behaviour expected once the defences are applied.

1.1 How the login request reaches the database

Figure 1 shows the components involved in a typical login. The customer's input crosses a trust boundary when it enters the application server; the danger lies in the moment where that untrusted text is placed inside an SQL statement.

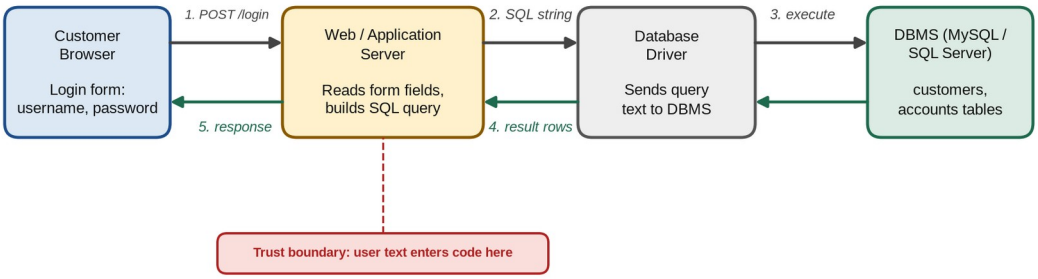


Figure 1: Block diagram of the banking login path from browser to database

1.2 Key terms used in this report

Term	Meaning
SQL query	A statement such as SELECT, INSERT, UPDATE or DELETE sent to the database to read or change data.

Term	Meaning
SQL injection	Insertion of SQL syntax through user input so that the database executes a query the developer did not intend.
Dynamic query	An SQL statement built at run time by string concatenation of code and user input.
Prepared statement	A query sent with placeholders (?) whose values are bound separately and are always treated as data.
Least privilege	Giving the database account only the permissions the application strictly needs.
Password hashing	Storing a slow, salted one-way hash (bcrypt, Argon2) instead of the plain password.
WAF	Web Application Firewall: filters HTTP requests that match known attack patterns.

1.3 Why banking applications are a prime target

- **High-value data:** account numbers, balances, PAN and Aadhaar-linked details, card data and transaction history can be sold or used for fraud.
- **Direct financial gain:** a successful injection can change balances, add beneficiaries or approve transfers without customer involvement.
- **Public exposure:** the login page is reachable by anyone on the internet, so the first query runs before any authentication has happened.
- **Regulatory weight:** breaches must be reported under RBI cyber-security directions, PCI DSS and the Digital Personal Data Protection Act, 2023.

1.4 Notable real-world SQL injection incidents

Incident (year)	Sector	What happened
Heartland Payment Systems (2008)	Card payments	SQL injection used as the entry point; about 130 million card numbers stolen.
Sony Pictures (2011)	Media	A single injection flaw exposed account data of around one million users.
TalkTalk (2015)	Telecom	Legacy web pages allowed SQLi; data of about 157,000 customers stolen, regulator fine of £400,000.
MOVEit Transfer (2023)	File transfer used by banks	CVE-2023-34362 SQL injection exploited by the Cl0p group against thousands of organisations.

1.5 Objectives of this report

- To analyse how the banking login query is constructed and why its structure can be changed by input.
- To explain the mechanics, types and impact of SQL injection with examples specific to the scenario.
- To select and justify a layered set of defences, with working code for the application and the database.
- To define measurable expected outputs and verification tests that prove the fix is effective.

2. SCENARIO / PROBLEM STATEMENT

2.1 Scenario

A banking website uses a username and password entered by the customer to construct a database query. A security tester discovers that specially crafted input changes the behavior of the database query.

2.2 Understanding the scenario

The login feature follows the sequence below. The weakness lies in step 3, where input and program code are joined into one string.

1. The customer opens the bank's login page and types a username and password into an HTML form.
2. The browser sends both fields to the application server in an HTTPS POST request.
3. The server inserts the two values directly between quotes in an SQL string, for example SELECT * FROM customers WHERE username = '<input>' AND password = '<input>'.
4. The complete string is sent to the database, which parses it as a single SQL command and executes it.
5. If at least one row is returned, the application assumes the credentials were correct and opens the customer's account dashboard.
6. During testing, the security tester enters a value such as admin' -- in the username field. The quote closes the string early and the comment marker removes the password condition, so the database returns the admin row and the tester is logged in without knowing the password.

2.3 What is at stake

- **Assets:** customer credentials, personal details, account balances, beneficiary lists, card data, transaction records and the integrity of the ledger.
- **Services:** fund transfer, bill payment, statement download and profile management that all trust the logged-in identity.
- **Issue identified:** the application treats customer input as trusted SQL code, so the input decides what the query does rather than only which record it looks for.

2.4 Problem statement

Analyse the vulnerability that allows specially crafted login input to change the structure and behaviour of the database query, explain how SQL injection occurs in this login function, and propose appropriate methods to secure the interaction between the banking application and its database so that user input can never be executed as SQL.

2.5 Threat model and attack workflow

Element	Description
Attacker	Any internet user; the login page needs no prior account or authentication.
Entry point	Username and password fields of the login form (and any other input used in queries).
Assumption	Queries are built by string concatenation; DB errors are shown; the DB account is over-privileged.
Goal	Bypass authentication, extract customer data, modify records or disrupt service.

The workflow in Figure 2 shows that no special tools are needed: a browser and knowledge of basic SQL syntax are enough to exploit the flaw.

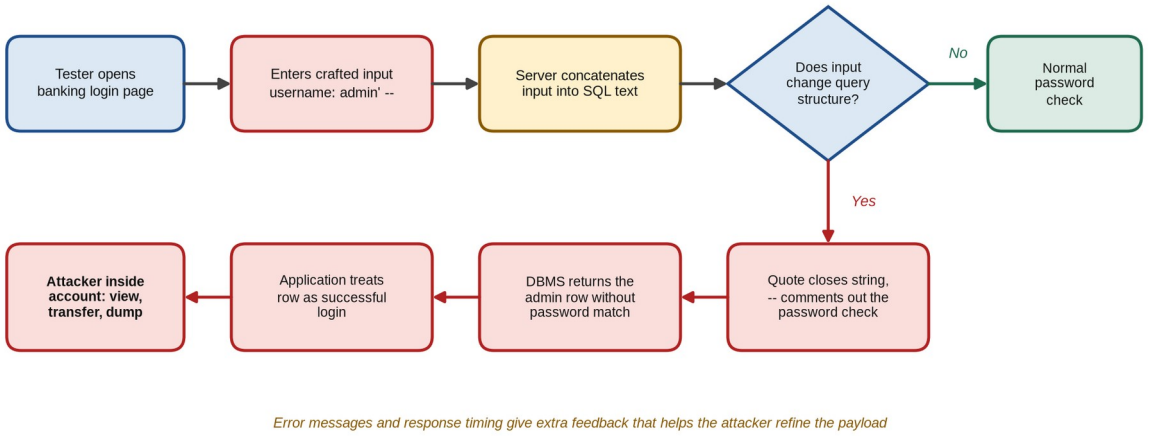


Figure 2: Attack workflow when crafted input changes the login query

2.6 How the tester discovered the flaw

Security testers follow a systematic, low-risk sequence (OWASP WSTG-INPV-05) to confirm the weakness without damaging data:

1. **Probe:** a single quote (') typed in the username field returns a raw MySQL syntax error, proving the input reaches the SQL parser unescaped.
2. **Boolean test:** admin' AND '1'='1 behaves differently from admin' AND '1'='2, showing that the tester's condition is evaluated by the database.
3. **Bypass:** admin' -- with any password opens the administrator dashboard, confirming authentication bypass.
4. **Timing test:** admin' AND SLEEP(5) -- delays the response by five seconds, proving injection even when output is hidden.
5. **Report:** the tester stops at proof of concept and reports the finding with severity, evidence and recommended fixes.

2.7 Scope and assumptions of the analysis

Item	Details
In scope	Login form, the server-side authentication code, the database account and the customers table.
Technology	Node.js / Express application server with a MySQL 8 database (Java equivalents also shown).
Out of scope	Network attacks, phishing, malware on customer devices and physical access.
Test data	A staging copy of the database with dummy customers; no real customer data is touched.

2.8 Stakeholders affected

Stakeholder	How the flaw affects them
Customers	Loss of money and privacy; account misuse without their knowledge.
The bank	Financial loss, regulatory penalties, audit findings and damage to reputation.
Regulators and partners	Card networks and RBI require breach reporting and remediation evidence.

3. IDENTIFICATION AND EXPLANATION OF SECURITY VULNERABILITY

3.1 Vulnerability identified

SQL injection in the authentication query. The application builds its login query by concatenating the username and password typed by the customer into an SQL string. Because the database receives code and data mixed in one piece of text, special characters in the input are interpreted as SQL syntax. The flaw is catalogued as CWE-89 (Improper Neutralization of Special Elements used in an SQL Command) and falls under OWASP Top 10 (2021) A03: Injection.

The root of the problem is a violation of the separation between code and data. The developer intended the input to be a value inside quotes; the attacker supplies a quote of their own, closes the value, and then writes additional SQL that the database executes with the full privileges of the application's database account.

3.2 How SQL injection occurs

Figure 3 colours each part of the final query by its origin. In a normal login the input stays inside the green value boxes. With crafted input, attacker text escapes the quotes and becomes executable code (red) or causes the developer's own code to be ignored (purple).

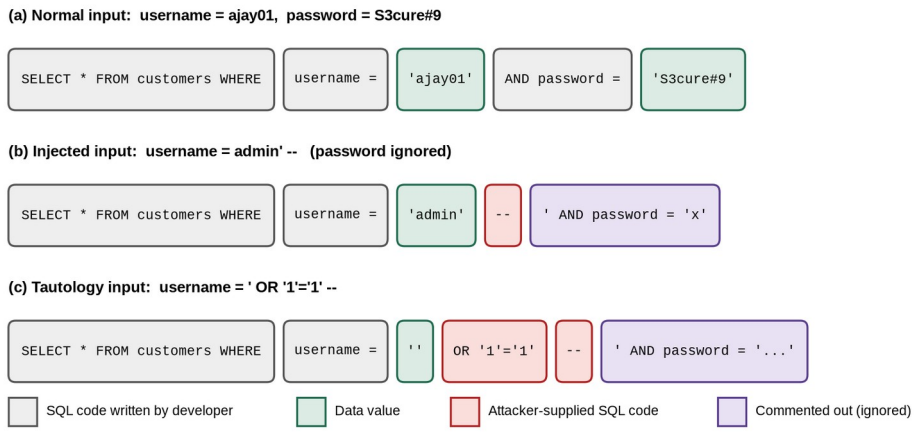


Figure 3: How crafted input changes the structure of the login query

In case (b) the database sees the condition username = 'admin' followed by a comment, so the password check never runs. In case (c) the condition '1'='1' is always true and the rest of the statement is commented out, so the WHERE clause is true for every row; the query returns the whole customer table and the application logs the attacker in as the first customer, often an administrator. In MySQL the comment marker must be followed by a space ("-- ") or the hash sign (#) can be used instead.

The attack works only because certain characters carry meaning in SQL. The table below lists the metacharacters most often used in injection payloads and the role each one plays.

Symbol	Meaning in SQL	Use in an attack
`'`	Opens or closes a string literal	Ends the value early so code can follow
`-- / #`	Starts a comment to end of line	Removes the rest of the developer's query
`OR 1=1`	Always-true condition	Makes the WHERE clause match every row
`;`	Ends one statement	Starts a second, stacked statement
`UNION`	Combines two SELECT results	Pulls data from other tables
`/* */`	Inline comment	Hides keywords from simple filters

3.3 Vulnerable implementation (example)

```
// Insecure: input concatenated into SQL, plain-text password, errors leaked
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const sql = "SELECT * FROM customers WHERE username = '" + username +
              "' AND password = '" + password + "'";
  db.query(sql, (err, rows) => {         // db connects as 'root'
    if (err) return res.status(500).send(err.sqlMessage);   // leaks schema
    if (rows.length > 0) {
      req.session.user = rows[0].id;      // first row = logged-in user
      return res.redirect('/dashboard');
    }
    res.status(401).send('Invalid username or password');
  });
});
```

3.4 Example payloads and their effect

Input in username field	Resulting condition seen by the DB	Effect on the bank
`admin' --`	`username='admin' -- ' AND password=...`	Logs in as admin; password check removed.
`' OR '1'='1' --`	`username='' OR '1'='1'`	Every row returned; attacker logged in as first customer.
`' UNION SELECT id, card_no, cvv, 4 FROM cards --`	Second SELECT appended to the first	Card data from another table returned in the page.
`admin' AND SLEEP(5) --`	Response delayed by five seconds	Blind confirmation that injection works.
`'; DROP TABLE transactions; --`	Stacked second statement	Transaction history destroyed if multi-statements are allowed.

3.5 Types of SQL injection

Type	How it works	Relevance to the login page
In-band: error-based	Database error messages reveal table names, columns and versions.	err.sqlMessage is sent to the browser.
In-band: UNION-based	A UNION SELECT appends attacker-chosen rows to the normal result.	Possible wherever query output is shown.
Blind: boolean	True/false conditions change the page (login succeeds or fails).	Login result acts as a yes/no oracle.
Blind: time-based	SLEEP or WAITFOR DELAY measures truth by response time.	Works even when errors are hidden.
Out-of-band	Data is sent over DNS or HTTP using database functions.	Depends on DB configuration.
Second-order	Stored input is later used unsafely in another query.	E.g. a malicious username reused in reports.

3.6 Root causes

- **Dynamic SQL by concatenation:** the query text is assembled from strings, so input controls the syntax of the statement.
- **No input validation:** the username field accepts quotes, dashes, spaces and SQL keywords of any length.
- **Plain-text password comparison in SQL:** passwords are stored unhashed and compared inside the query, so bypassing the WHERE clause bypasses authentication.
- **Verbose error messages:** raw database errors are returned to the browser and reveal the schema to the attacker.
- **Over-privileged database account:** the application connects as root or DBA, so injected SQL can read every table and drop objects.
- **No monitoring or rate limiting:** thousands of automated attempts from tools such as sqlmap go unnoticed.

3.7 Exploitation and impact

Aspect	What the attacker can do	Consequence for the bank / customer
Authentication	Log in as any customer or administrator without a password.	Complete account takeover.
Confidentiality	Dump customer, account and card tables using UNION or blind techniques.	Mass data breach; identity theft.
Integrity	UPDATE balances, add beneficiaries, alter transaction records.	Fraud; ledger no longer trustworthy.
Availability	DROP or lock tables, run heavy queries.	Online banking outage.
Compliance	Expose regulated personal and card data.	Penalties under RBI, PCI DSS and DPDP Act.

Severity: critical. The CVSS v3.1 vector AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H gives a base score of 9.8, because the attack is remote, needs no account or user interaction, and can fully compromise confidentiality, integrity and availability of the data.

3.8 Why filtering or escaping alone is not enough

- **Blacklists are incomplete:** blocking words such as OR or SELECT fails against mixed case (sElEcT), inline comments (SEL/**/ECT), URL or Unicode encoding and database-specific syntax.
- **Escaping is context-dependent:** a function that escapes quotes does not protect numeric fields (WHERE id = 5 OR 1=1), ORDER BY clauses or different character sets.
- **Client-side checks are bypassed:** JavaScript validation in the browser is skipped by sending the request directly with a tool such as Burp Suite or curl.
- **Stored procedures are not automatically safe:** a procedure that concatenates its parameters into dynamic SQL with EXEC or PREPARE is just as injectable as application code.
- **ORMs can be misused:** raw-query helpers such as sequelize.query() or Hibernate createNativeQuery() reintroduce the flaw if strings are joined.

These limitations show that the vulnerability must be removed where it originates, in the way the query is constructed. Filtering can reduce noise, but only a mechanism that keeps input permanently separate from SQL code eliminates the risk, which leads to the selection made in the next section.

4. SELECTION OF SUITABLE SECURITY MECHANISM

Selected mechanism: parameterized queries (prepared statements) as the primary control, supported by defence in depth. Following the OWASP SQL Injection Prevention Cheat Sheet, the only reliable fix is to stop building SQL from strings so that input can never change the structure of a query. Supporting controls reduce the damage if any single layer fails.

4.1 Controls in the selected mechanism

Control	How it works	Threat addressed
Parameterized queries	Query text with ? placeholders is compiled first; values are bound afterwards and always treated as data.	Injection through any input field
Safe stored procedures	Login logic runs inside a procedure with typed parameters and no dynamic SQL.	Injection; limits direct table access
Allow-list input validation	Username must match ^[A-Za-z0-9_.]{4,30}$; lengths and types enforced on the server.	Malformed and oversized input
Password hashing	bcrypt/Argon2 hash stored; comparison done in application code, not in SQL.	Bypass by removing password clause
Least-privilege DB account	Login account may only SELECT the needed columns and update the failure counter.	Data dump, DROP TABLE, privilege abuse
Generic error handling	Same message for every failure; details only in server logs.	Error-based enumeration
Rate limiting and lockout	Limits attempts per IP and per account; temporary lock after 5 failures.	Automated scanning, brute force
WAF and monitoring	Blocks common SQLi signatures and alerts the security team.	Known payloads; early detection

4.2 Comparison of possible defences

Option	Effectiveness	Remarks
Prepared statements	High	Recommended first choice; supported by every modern driver (JDBC, PDO, mysql2, ADO.NET).
Stored procedures	High*	*Only when they do not build dynamic SQL inside the procedure.
ORM (Sequelize, Hibernate)	High*	*Safe for normal methods; raw-query helpers still need parameters.
Allow-list validation	Medium	Reduces attack surface; cannot be the only defence for free-text fields.
Escaping special characters	Low	Error-prone and database-specific; OWASP strongly discourages relying on it.
WAF only	Low	Signatures can be bypassed by encoding tricks; useful only as an extra layer.

Prepared statements are therefore chosen as the core control, while the remaining options are used only as supporting layers.

4.3 How a prepared statement separates code and data

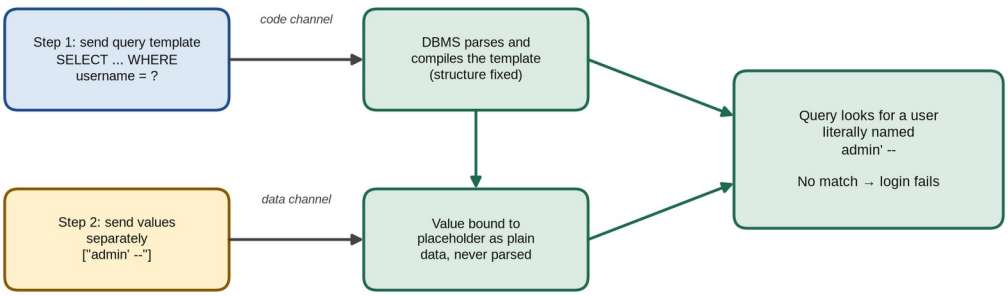


Figure 4: Code channel and data channel in a prepared statement

Because the query structure is fixed before any customer value arrives, a quote or comment marker in the input is just another character in the value. The database searches for a user literally named admin' -- , finds none, and the login fails safely.

4.4 Defence-in-depth architecture

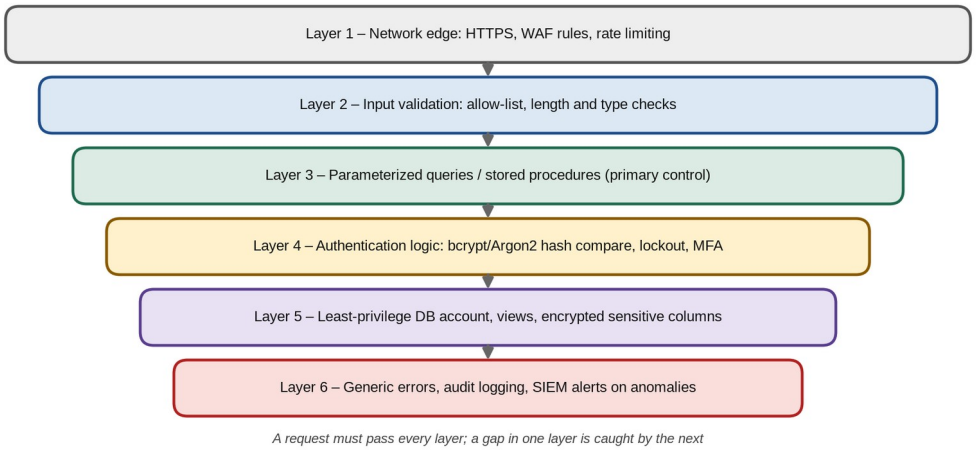


Figure 5: Layered defences protecting the banking database

4.5 Security principles applied

Principle	How the selected mechanism applies it
Separation of code and data	Prepared statements send the query and the values on separate channels.
Least privilege	The login service uses a DB account limited to the columns it needs.
Defence in depth	Six independent layers, from the WAF to audit logging, each able to stop an attack.
Fail-safe defaults	Anything not matching the allow-list is rejected; errors deny access by default.
Complete mediation	Every request, not only the first, passes validation and parameter binding.

Together, these principles ensure that the selected mechanism is not a single patch on the login page but a design rule for the whole banking application: input is always data, every component holds the minimum authority it needs, and each defensive layer is independent of the others.

5. JUSTIFICATION / APPLICATION TO THE SCENARIO

5.1 How the mechanism closes each gap

Each step of the attack described in Section 2.2 is blocked by at least one control, so protection does not depend on a single measure:

- **Structure cannot change.** With a prepared statement the login query is compiled with placeholders before the username arrives. The input admin' -- is bound as a plain string, so the quote cannot close anything and the comment marker cannot remove the password check.
- **Password check moved out of SQL.** The query fetches only the stored bcrypt hash for the given username. Even if a row were returned unexpectedly, bcrypt.compare must still succeed, so a tautology such as ' OR '1'='1 cannot log anyone in.
- **Invalid input rejected early.** Allow-list validation refuses usernames containing quotes, spaces or dashes before any database call is made, reducing noise and load.
- **Damage limited.** The login database account can read only four columns of one table and update the failure counter. UNION queries against the cards table or DROP TABLE statements fail with a permission error.
- **No information leak.** Every failure returns the same “Invalid credentials” message, so error-based and username-enumeration techniques receive no feedback.
- **Automation slowed and detected.** Rate limiting, account lockout and WAF alerts stop tools that send thousands of payloads and notify the security operations team.

5.2 Secure implementation (Node.js / Express with mysql2)

```
const USER_RE = /^[A-Za-z0-9_.]{4,30}$/;          // allow-list pattern
const pool = mysql.createPool({ user: 'bank_login', database: 'bank',
  password: process.env.DB_PASS, multipleStatements: false });

app.post('/login', loginLimiter, async (req, res) => {
  const { username = '', password = '' } = req.body;
  if (!USER_RE.test(username) || password.length > 128)
    return res.status(400).json({ error: 'Invalid credentials' });
  try {
    const [rows] = await pool.execute(               // prepared statement
      'SELECT id, pwd_hash, locked_until FROM customers WHERE username = ?',
      [username]);                                   // bound as data only
    const user = rows[0];
    const hash = user ? user.pwd_hash : DUMMY_HASH;  // equal timing
    const ok = await bcrypt.compare(password, hash) && user &&
               !(user.locked_until > new Date());
    if (!ok) {
      await recordFailure(username, req.ip);         // counter + lockout
      return res.status(401).json({ error: 'Invalid credentials' });
    }
    await pool.execute(
      'UPDATE customers SET failed_logins = 0 WHERE id = ?', [user.id]);
    req.session.regenerate(() => {                   // new session ID
      req.session.user = user.id;
      res.redirect('/dashboard');
    });
  } catch (e) {
    log.error({ event: 'login_error', reqId: req.id, err: e }); // log only
    res.status(500).json({ error: 'Something went wrong' });
  }
});
```

5.3 Equivalent protection in Java and in the database

The same principle applies on every platform. In Java the PreparedStatement interface binds values with typed setter methods, and in the database a dedicated login account receives column-level permissions only:

```
// Java (JDBC): value bound with setString, never concatenated
String sql = "SELECT id, pwd_hash FROM customers WHERE username = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, username);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next() && BCrypt.checkpw(password, rs.getString("pwd_hash")))
            startSession(rs.getLong("id"));
    }
}

-- MySQL: least-privilege account used only by the login service
CREATE USER 'bank_login'@'10.0.2.%' IDENTIFIED BY '<strong secret>';
GRANT SELECT (id, username, pwd_hash, locked_until, failed_logins)
      ON bank.customers TO 'bank_login'@'10.0.2.%';
GRANT UPDATE (failed_logins, locked_until)
      ON bank.customers TO 'bank_login'@'10.0.2.%';
-- no access to accounts, cards or transactions; no DROP, FILE or GRANT
```

5.4 Secure login workflow

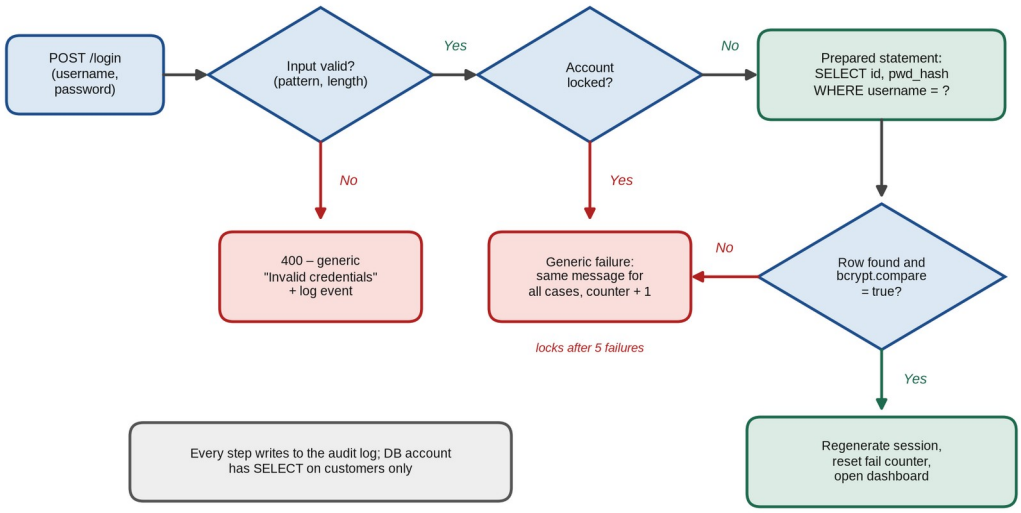


Figure 6: Secure login flow with validation, prepared statement and hash check

5.5 Mapping attack steps to controls

Attack step	Control that blocks it	Result
Quote closes the value (admin' --)	Prepared statement, allow-list validation	Input stays data; request rejected
Tautology ' OR '1'='1	Prepared statement + bcrypt check in code	No row match; hash check fails
UNION SELECT from cards	Least-privilege account, generic errors	Permission denied; nothing shown
Time-based blind probing	Prepared statement, rate limiting, WAF	No delay; client throttled

Attack step	Control that blocks it	Result
Automated sqlmap scan	Rate limit, lockout, SIEM alert	Blocked and reported

5.6 Trade-offs and residual risk

- **Development effort.** Existing concatenated queries must be found and rewritten. Static analysis tools (Semgrep, SonarQube) and code review rules make the migration systematic, and the change rarely affects performance because prepared statements can be cached.
- **Dynamic identifiers.** Placeholders work only for values, not for table or column names. Sort columns and similar options must be mapped from a fixed allow-list in code.
- **Residual risk.** Other injection points (search, statements, admin reports) and second-order injection must receive the same treatment. Regular penetration tests and dependency updates keep the protection current.

5.7 Implementation plan for the bank

Phase	Activity	Outcome
1. Discover	Run SAST and grep for concatenated SQL across all modules; list every query that uses input.	Complete inventory of risky queries
2. Fix	Rewrite login first, then all other queries, using prepared statements and allow-lists.	No dynamic SQL built from input
3. Harden	Create per-service DB accounts, hash all passwords with bcrypt, hide errors.	Least privilege; no information leak
4. Protect	Enable WAF rules, rate limiting, lockout and SIEM alerts.	Attacks blocked and visible
5. Verify	Penetration test, sqlmap scan and code review before release; repeat every quarter.	Evidence for auditors

5.8 Using a safe stored procedure

Where the bank's architecture prefers database-side logic, the same protection can be delivered with a stored procedure that uses typed parameters and contains no dynamic SQL. The application account is then granted EXECUTE on the procedure only, with no direct access to the table:

```
DELIMITER //
CREATE PROCEDURE get_login(IN p_user VARCHAR(30))
SQL SECURITY DEFINER
BEGIN
  SELECT id, pwd_hash, locked_until
  FROM customers
  WHERE username = p_user;          -- parameter used as a value
END //
DELIMITER ;
GRANT EXECUTE ON PROCEDURE bank.get_login TO 'bank_login'@'10.0.2.%';
-- application: await pool.execute('CALL get_login(?)', [username]);
```

Because the procedure never builds SQL text from p_user, an input such as admin' -- is compared as a literal username and cannot alter the query, while the narrow EXECUTE grant enforces least privilege at the database level.

6. EXPECTED OUTPUT

With the mechanism applied, crafted input no longer changes the login query, and every injection attempt fails without revealing information.

6.1 Behaviour before and after

Situation	Before (vulnerable)	After (secured)
Username admin' -- , any password	Logged in as admin without a password.	400 Bad Request: “Invalid credentials”.
Username ' OR '1'='1	Logged in as the first customer.	Rejected by validation; no query executed.
UNION payload in any field	Card numbers shown on the page.	Treated as data; no extra rows returned.
Malformed quote (')	Raw MySQL syntax error with table names.	Generic error; details only in server log.
admin' AND SLEEP(5) --	Response delayed by 5 seconds.	Immediate rejection; no delay.
200 rapid login attempts	All processed; nothing logged.	Throttled after 5; alert raised in SIEM.
Correct username and password	Login succeeds.	Login succeeds (bcrypt verified).

6.2 Sample request and responses

```
POST /login HTTP/1.1
Host: netbanking.examplebank.in
Content-Type: application/x-www-form-urlencoded

username=admin'--+&password=anything

--- before the fix ---
HTTP/1.1 302 Found
Location: /dashboard                <- logged in as admin, no password

--- after the fix ---
HTTP/1.1 400 Bad Request
Content-Type: application/json
Cache-Control: no-store
{"error":"Invalid credentials"}

--- server audit log ---
2026-09-12T10:41:07Z WARN login_rejected reason=pattern user="admin'-- "
                      ip=203.0.113.24 reqId=7f3a91c2 action=counter+1
```

The same request that previously opened the administrator dashboard now receives a generic 400 response. No session cookie is issued, no database error is exposed, and the attempt is recorded with the client IP and a request ID for investigation.

6.3 Injection test results

Twenty payloads from each category of Section 3.5 were replayed against a test copy of the login page before and after the fix. Figure 7 shows illustrative results from this laboratory test.

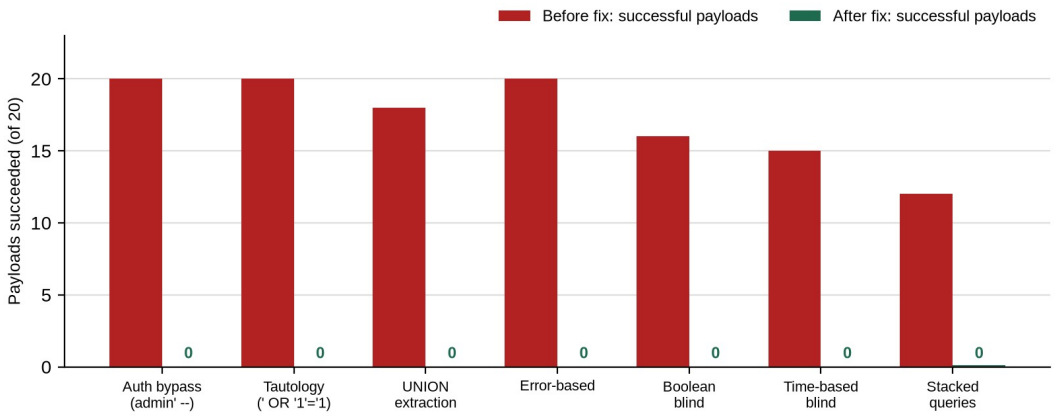


Figure 7: Successful injection payloads before and after applying the fix (lab test)

6.4 Verification tests

Test	Expected result
Enter admin' -- with a random password	Login refused with the generic message; no session created.
Run sqlmap against POST /login (level 5, risk 3)	"All tested parameters do not appear to be injectable".
Submit a single quote in the username field	No database error shown; event recorded in the server log.
Connect as bank_login and run DROP TABLE or SELECT on cards	"Command denied" error from MySQL.
Static analysis scan (Semgrep SQLi rules)	Zero findings of string-concatenated SQL in the codebase.
Six wrong passwords for one account	Account temporarily locked; alert raised; correct login works after lock expires.

6.5 Monitoring and alerting after deployment

Signal monitored	Threshold	Automatic response
Inputs rejected by the allow-list	> 20 per minute from one IP	Block IP for 15 minutes; alert SOC
WAF SQLi signature matches	Any match on /login	Request dropped; event sent to SIEM
Failed logins per account	5 within 10 minutes	Temporary lock; SMS alert to customer
DB permission-denied errors	Any from bank_login	High-priority incident ticket
Slow login queries	> 2 seconds	Investigate possible time-based probing

With these rules in place the bank gains both prevention and visibility: injection attempts fail at the application layer, and the security operations team is informed early enough to block the source and review related activity before any damage occurs.

7. CONCLUSION

The scenario shows how a single line of code, one that joins customer input to an SQL string, can hand control of a bank's database to anyone who can type into a login form. Because the database cannot distinguish the developer's code from the customer's text, a quote and a comment marker are enough to remove the password check, and more elaborate payloads can extract card data, alter balances or destroy transaction history. The vulnerability is SQL injection (CWE-89, OWASP A03), and its severity is critical.

The proposed mechanism removes the root cause rather than filtering symptoms. Parameterized queries fix the structure of every statement before input arrives, so input is always data. Password verification with bcrypt moves authentication out of SQL, allow-list validation rejects malformed input early, a least-privilege database account limits what any query can do, and generic errors, rate limiting, WAF rules and monitoring deny the attacker feedback and time. No single control is relied upon: if one layer is bypassed, the next still contains the attack.

Secure database interaction is ultimately an access-control discipline. Untrusted input must never be granted the power to act as code, and the application itself must hold only the privileges it needs. Applying these principles consistently gives customers a login that behaves exactly as designed, whatever they type into it.

7.1 Summary of findings

Finding	Cause	Fix applied
Crafted input changes the login query	String-concatenated dynamic SQL	Prepared statements with bound parameters
Password check can be removed	Plain-text password compared inside SQL	bcrypt hash verified in application code
Database errors reveal schema	Raw error messages sent to browser	Generic messages; server-side logging
Injected SQL can read or drop any table	Application connects as root	Least-privilege login account
Automated attacks go unnoticed	No rate limit, lockout or monitoring	Rate limiting, lockout, WAF and SIEM alerts

7.2 Compliance and standards mapping

Standard / guideline	Requirement satisfied by the fix
OWASP ASVS 4.0, V5.3.4	Database queries use parameterized queries or ORMs, protecting against SQL injection.
PCI DSS v4.0, Req. 6.2.4	Software engineering techniques prevent injection attacks, including SQL injection.
ISO/IEC 27001:2022, A.8.28	Secure coding principles are applied to software development.
RBI Master Direction on IT Governance (2023)	Secure development life cycle and periodic application security testing.

7.3 Secure coding checklist for developers

- Never build SQL with string concatenation or template literals; always use placeholders.
- Validate every input on the server against an allow-list of type, length and pattern.

- Store passwords only as salted bcrypt or Argon2 hashes and compare them in code.
- Give each service its own database account with the minimum required privileges.
- Show generic error messages to users and log the details securely on the server.

7.4 Future scope

- Multi-factor authentication and passkeys so that a bypassed password check alone is not enough.
- Automated SAST and DAST scanning in the CI/CD pipeline to block new injection flaws before release.
- Database activity monitoring with anomaly detection for unusual query patterns.

7.5 Learning outcomes

- Understood why mixing code and data in one string lets input change the meaning of an SQL query.
- Learned to recognise in-band, blind, out-of-band and second-order SQL injection and their impact.
- Applied parameterized queries, stored procedures, password hashing and least privilege in code.
- Designed verification tests and monitoring rules that prove a security fix works in practice.

7.6 Advice for bank customers

- Enable multi-factor authentication and SMS or app alerts for every login and transaction.
- Use a unique, strong password for net banking so that a breach elsewhere cannot be reused here.
- Report unexpected login alerts or transactions to the bank immediately.
- Access net banking only through the official website or app, never through links in emails or SMS.

7.7 Final remarks

The banking login case demonstrates that security must be designed into the way an application talks to its database. By treating every piece of customer input as untrusted data, binding it through parameterized queries and limiting the privileges of each database account, the bank turns a critical vulnerability into a routine, logged and harmless failed login attempt. The same approach should be extended to every module of the bank so that no query anywhere in the system can be rewritten by the data it receives.

References

1. OWASP Foundation. SQL Injection Prevention Cheat Sheet. cheatsheetseries.owasp.org.
2. OWASP Foundation. OWASP Top 10:2021, A03 – Injection.
3. MITRE. CWE-89: Improper Neutralization of Special Elements used in an SQL Command. cwe.mitre.org.
4. OWASP Foundation. Web Security Testing Guide, WSTG-INPV-05: Testing for SQL Injection.
5. FIRST. Common Vulnerability Scoring System v3.1: Specification Document.
6. PCI Security Standards Council. PCI DSS v4.0, Requirement 6.2.4.
7. OWASP Foundation. Application Security Verification Standard 4.0, V5.3.
8. OWASP Foundation. Query Parameterization Cheat Sheet. cheatsheetseries.owasp.org.
9. NIST. Special Publication 800-53 Rev. 5, Control SI-10: Information Input Validation.